

- Description

This module manages a smart toll booth traffic monitoring system, ensuring that:

1. **The number of vehicles passing through the toll station is continuously monitored.**
2. **If traffic congestion exceeds a predefined threshold, additional toll lanes are opened to improve flow.**
3. **If traffic volume drops below a minimum threshold, some lanes are closed to optimize resource usage.**
4. **A log of all vehicle counts, congestion alerts, and lane adjustments is maintained for operational efficiency.**

This system helps prevent toll station bottlenecks, optimize lane usage, and improve overall traffic efficiency.

- Specification

module SmartTollBoothMonitoringEx

var currentTraffic, maxTraffic, minTraffic, openLanes: integer

var log: sequence of string

process vehicleArrives(numVehicles: integer)

functional scenario form:

currentTraffic <= maxTraffic && log = log $\oplus$ ("Low Traffic: Closing Unused Lanes."), Vehicles entered, lanes adjusted.

||

currentTraffic > maxTraffic && log = log $\oplus$ ("Heavy Congestion Alert: Additional Lanes Opened."), Heavy congestion warning triggered, opening additional lanes.

end process

process vehicleExits(numVehicles: integer)

functional scenario form:

log = log $\oplus$ ("Exit: " + numVehicles + " vehicles | New Traffic: " + currentTraffic + " | Open Lanes: " + openLanes), Vehicles exited, lanes adjusted.

end process

end module

- Java Code

```
package code;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ThreadLocalRandom;
/**
 * c124_SmartTollBoothSystemEx
 *
 * SmartWell structure:
 * - Core domain model (TollBooth, Lane, VehicleClass, TagAccount)
```

```

* - Policies (LanePolicy, PricingPolicy, Surge, OffPeak, SafetyMinLane
s)
* - Payments (PaymentProcessor, PricingEngine)
* - Sensors (FlowSensor, SpeedSensor, QueueSensor)
* - Controller (TrafficController) with peak-hour bias and anti-thrash
* - Alerts & Logging (AlertType, Alert, BoothLogger)
* - Stats (BoothStats)
* - Scenarios (VT/FT/Z3/UVT) using your sequences
* - Runner & CLI harness: MODE=VT|FT|Z3|UVT, SEED=<long>, LOG=on|off,
PRICING=on|off
*
* Notes:
* - Negative arrivals/exits are treated as abnormal; we log alerts but
still apply baseline math (as your examples do).
* - Lane adjustment mirrors your thresholds and adds soft rules (safet
y floor, anti-thrash).
* - Dynamic pricing is optional (PRICING=on) and prints charge samples
for demo; it does NOT block traffic flow.
* - All comments are in English as requested.
*/
public class c124_SmartTollBoothSystemEx {
    /* =====
    * ===== Alerts/Logger =====
    * ===== */
    enum AlertType {
        CONGESTION,
        LOW_TRAFFIC,
        NEGATIVE_ARRIVAL,
        NEGATIVE_EXIT,
        LANE_POLICY_SWITCH,
        SENSOR_FAULT,
        PAYMENT_ERROR,
        INFO
    }
    static final class Alert {
        final AlertType type;
        final String message;
        final long ts;
        Alert(AlertType type, String message) {
            this.type = type;
            this.message = message;
            this.ts = System.currentTimeMillis();
        }
    }
}

```

```

        @Override public String toString() { return "[" + type + "]" + " " +
message + " @ " + ts; }
    }
    static final class BoothLogger {
        private final List<String> lines = new ArrayList<>();
        private final List<Alert> alerts = new ArrayList<>();
        private boolean console = true;
        void setConsole(boolean on) { this.console = on; }
        void log(String s) { lines.add(s); if (console) System.out.println(s); }
        void alert(AlertType t, String msg) {
            Alert a = new Alert(t, msg);
            alerts.add(a);
            if (console) System.out.println("ALERT: " + a);
        }
        List<String> lines() { return Collections.unmodifiableList(lines); }
        List<Alert> alerts() { return Collections.unmodifiableList(alerts); }
        void printSummary() {
            log("\n=== Logger Summary ===");
            log("Lines: " + lines.size() + ", Alerts: " + alerts.size());
        }
        Map<AlertType, Long> cnt = new EnumMap<>(AlertType.class);
        for (Alert a : alerts) cnt.put(a.type, cnt.getOrDefault(a.type, 0L) + 1);
        for (AlertType t : AlertType.values()) log(" - " + t + ": " + cnt.getOrDefault(t, 0L));
    }
}
/* =====
 * ===== Core Model =====
 * ===== */
enum VehicleClass {
    CAR(1.0), TRUCK(2.0), BUS(1.5), MOTORCYCLE(0.7);
    final double weight;
    VehicleClass(double w) { this.weight = w; }
}
static final class TagAccount {
    final String tagId;
    double balance;
    TagAccount(String tagId, double initial) { this.tagId = tagId;
this.balance = initial; }

```

```

        boolean debit(double amt) { if (balance >= amt) { balance -
= amt; return true; } return false; }
    }
    static final class Lane {
        final int id;
        boolean open;
        boolean ezPassOnly;
        int queue; // simple queue length
        Lane(int id, boolean open, boolean ezPassOnly) {
            this.id = id; this.open = open; this.ezPassOnly = ezPassOnly; this.queue = 0;
        }
    }
    /** Policy object that computes desired open lanes based on thresholds and safety rules. */
    static final class LanePolicy {
        final double highRatio; // e.g., traffic > 80% of max -> many lanes
        final int highOpen; // e.g., 8
        final int normalOpen; // e.g., 5
        final int lowOpen; // e.g., 3
        final int safetyMin; // never go below this number
        final int antiThrashWindow; // minimum minutes before switching again
        LanePolicy(double highRatio, int highOpen, int normalOpen, int lowOpen, int safetyMin, int antiThrashWindow) {
            this.highRatio = highRatio;
            this.highOpen = highOpen;
            this.normalOpen = normalOpen;
            this.lowOpen = lowOpen;
            this.safetyMin = safetyMin;
            this.antiThrashWindow = antiThrashWindow;
        }
        int decide(int maxTraffic, int minTraffic, int currentTraffic)
        {
            if (maxTraffic <= 0) return Math.max(safetyMin, normalOpen);
            ;
            double ratio = (double) currentTraffic / (double) maxTraffic;
            if (ratio > highRatio) return Math.max(safetyMin, highOpen);
            ;
            if (currentTraffic < minTraffic) return Math.max(safetyMin, lowOpen);
            return Math.max(safetyMin, normalOpen);
        }
    }

```

```

    }
}
/** Pricing parameters. */
static final class PricingPolicy {
    final double basePrice;           // base per-car fee
    final double classMultiplier;    // additional multiplier per vehicle class weight
    final double surgeK;              // surge coefficient
    final double offPeakDiscount;     // discount during off-peak
    final int peakStartMin;           // e.g., 7:00 -> 420
    final int peakEndMin;             // e.g., 9:30 -> 570
    final int evePeakStartMin;        // e.g., 17:00 -> 1020
    final int evePeakEndMin;          // e.g., 19:30 -> 1170
    PricingPolicy(double basePrice, double classMultiplier, double surgeK, double offPeakDiscount,
                  int peakStartMin, int peakEndMin, int evePeakStartMin, int evePeakEndMin) {
        this.basePrice = basePrice;
        this.classMultiplier = classMultiplier;
        this.surgeK = surgeK;
        this.offPeakDiscount = offPeakDiscount;
        this.peakStartMin = peakStartMin;
        this.peakEndMin = peakEndMin;
        this.evePeakStartMin = evePeakStartMin;
        this.evePeakEndMin = evePeakEndMin;
    }
    boolean isPeak(int minuteOfDay) {
        int m = minuteOfDay % (24 * 60);
        return (m >= peakStartMin && m <= peakEndMin) || (m >= evePeakStartMin && m <= evePeakEndMin);
    }
}

/** Aggregated statistics for one run. */
static final class BoothStats {
    long totalArrivals;
    long totalExits;
    long negativeArrivals;
    long negativeExits;
    long laneSwitches;
    long congestionAlerts;
    long lowTrafficAlerts;
    int maxObservedTraffic;
    int minObservedTraffic;
    double totalRevenue;
}

```

```

        BoothStats(initial) {
            this.maxObservedTraffic = initial;
            this.minObservedTraffic = initial;
        }
        void onTraffic(int current) {
            maxObservedTraffic = Math.max(maxObservedTraffic, current);
            minObservedTraffic = Math.min(minObservedTraffic, current);
        }
        void print(BoothLogger log) {
            log.log("\n=== Booth Stats ===");
            log.log("Total Arrivals: " + totalArrivals);
            log.log("Total Exits: " + totalExits);
            log.log("Negative Arrivals: " + negativeArrivals);
            log.log("Negative Exits: " + negativeExits);
            log.log("Lane Switches: " + laneSwitches);
            log.log("Congestion Alerts: " + congestionAlerts);
            log.log("Low-Traffic Alerts: " + lowTrafficAlerts);
            log.log("Max Observed Traffic: " + maxObservedTraffic);
            log.log("Min Observed Traffic: " + minObservedTraffic);
            log.log(String.format(Locale.ROOT, "Total Revenue: %.2f", totalRevenue));
        }
    }

    /** Core booth model (preserves your public API and messages). */
    static final class TollBooth {
        private final int maxTraffic;
        private final int minTraffic;
        private int currentTraffic;
        private int openLanes;
        private final List<String> log;
        // SmartWell extensions:
        private final BoothLogger sharedLogger;
        private final LanePolicy lanePolicy;
        private final List<Lane> lanes = new ArrayList<>();
        private final Map<String, TagAccount> accounts = new ConcurrentHashMap<>();

        private final PricingEngine pricingEngine;
        private final BoothStats stats;
        private int lastPolicyChangeMinute = Integer.MIN_VALUE;

        TollBooth(int maxTraffic, int minTraffic, int initialTraffic,
                  LanePolicy lanePolicy, PricingEngine pricing, BoothLogger logger) {
            this.maxTraffic = maxTraffic;
            this.minTraffic = minTraffic;

```

```

        this.currentTraffic = initialTraffic;
        this.openLanes = 5;
        this.log = new ArrayList<>();
        this.sharedLogger = logger;
        this.lanePolicy = lanePolicy;
        this.pricingEngine = pricing;
        this.stats = new BoothStats(initialTraffic);
        // Create initial 8 lanes, default 5 open (lane 0..4), lane
s 5..7 closed
        for (int i = 0; i < 8; i++) {
            boolean open = i < 5;
            boolean ezPass = (i == 0 || i == 1); // two EZPass pref
erred lanes
            lanes.add(new Lane(i, open, ezPass));
        }
    }
    int getCurrentTraffic() { return currentTraffic; }
    int getMaxTraffic() { return maxTraffic; }
    int getMinTraffic() { return minTraffic; }
    int getOpenLanes() { return openLanes; }
    BoothStats stats() { return stats; }
    TagAccount openAccount(String tagId, double initial) {
        TagAccount acc = new TagAccount(tagId, initial);
        accounts.put(tagId, acc);
        return acc;
    }
    /** Public API: arrival; mirrors your baseline message and beha
vior. */
    public void vehicleArrives(int numVehicles) {
        vehicleArrives(numVehicles, null, VehicleClass.CAR, -1);
    }
    /** Extended arrival with optional tag and class; minuteOfDay f
or pricing demo. */
    public void vehicleArrives(int numVehicles, String ezTag, Vehic
leClass vClass, int minuteOfDay) {
        if (numVehicles < 0) {
            sharedLogger.alert(AlertType.NEGATIVE_ARRIVAL, "Negativ
e arrival request: " + numVehicles);
            stats.negativeArrivals++;
        }
        currentTraffic += numVehicles;
        adjustLanes(minuteOfDay);
        String message = String.format("Vehicles Arrived: %d | Curr
ent Traffic: %d | Open Lanes: %d",

```

```

        numVehicles, currentTraffic, openLanes);
    if (currentTraffic > maxTraffic) {
        message += " | WARNING: Heavy Congestion! Opening Addit
ional Lanes.";
        stats.congestionAlerts++;
        sharedLogger.alert(AlertType.CONGESTION, "Traffic=" + c
urrentTraffic + " > max=" + maxTraffic);
    } else if (currentTraffic < minTraffic) {
        message += " | Traffic Low. Closing Unused Lanes.";
        stats.lowTrafficAlerts++;
        sharedLogger.alert(AlertType.LOW_TRAFFIC, "Traffic=" +
currentTraffic + " < min=" + minTraffic);
    }
    sharedLogger.log(message);
    log.add(message);
    stats.totalArrivals += Math.max(0, numVehicles);
    stats.onTraffic(currentTraffic);
    // Optional: demo a few tag charges at arrival time (no blo
cking).

    if (pricingEngine != null && minuteOfDay >= 0) {
        double fee = pricingEngine.quote(minuteOfDay, vClass, c
urrentTraffic, maxTraffic, openLanes);
        if (ezTag != null) {
            TagAccount acc = accounts.get(ezTag);
            if (acc != null) {
                boolean ok = acc.debit(fee);
                if (!ok) sharedLogger.alert(AlertType.PAYMENT_E
RROR,
                    "Insufficient balance for tag=" + ezTag
+ ", need=" + fee);
            } else {
                stats.totalRevenue += fee;
                sharedLogger.log(String.format(Locale.ROOT,
                    "[Pricing] ezTag=%s charged %.2f, n
ew balance=%.2f",
                    ezTag, fee, acc.balance));
            }
        } else {
            sharedLogger.alert(AlertType.PAYMENT_ERROR, "Un
known tag=" + ezTag + " at arrival.");
        }
    } else {
        stats.totalRevenue += fee; // treat as cash lane fo
r demo

```

```

        sharedLogger.log(String.format(Locale.ROOT,
            "[Pricing] Cash vehicle charged %.2f", fee)
    );
    }
}

/** Public API: exit; mirrors your baseline message and behavior. */
public void vehicleExits(int numVehicles) {
    vehicleExits(numVehicles, -1);
}

/** Extended exit with minuteOfDay for symmetry (not used in pricing now). */
public void vehicleExits(int numVehicles, int minuteOfDay) {
    if (numVehicles < 0) {
        sharedLogger.alert(AlertType.NEGATIVE_EXIT, "Negative exit request: " + numVehicles);
        stats.negativeExits++;
    }
    currentTraffic = Math.max(0, currentTraffic - numVehicles);
    adjustLanes(minuteOfDay);
    String line = String.format("Vehicles Exited: %d | Current Traffic: %d | Open Lanes: %d",
        numVehicles, currentTraffic, openLanes);
    sharedLogger.log(line);
    log.add(String.format("Exit: %d vehicles | New Traffic: %d | Open Lanes: %d",
        numVehicles, currentTraffic, openLanes));
    stats.totalExits += Math.max(0, numVehicles);
    stats.onTraffic(currentTraffic);
}

/** Policy + anti-thrash + lane open/close bookkeeping. */
private void adjustLanes(int minuteOfDay) {
    int desired = lanePolicy.decide(maxTraffic, minTraffic, currentTraffic);
    // Anti-thrash: only allow changes if enough minutes passed
    if (minuteOfDay >= 0 && (minuteOfDay - lastPolicyChangeMinute) < lanePolicy.antiThrashWindow) {
        desired = openLanes; // hold
    }
    if (desired != openLanes) {
        applyLaneCount(desired);
        openLanes = desired;
        stats.laneSwitches++;
    }
}

```

```

        lastPolicyChangeMinute = minuteOfDay >= 0 ? minuteOfDay
: lastPolicyChangeMinute;
        sharedLogger.alert(AlertType.LANE_POLICY_SWITCH,
            "Switch lanes -> " + desired + " at traffic=" +
currentTraffic);
    }
}
private void applyLaneCount(int desiredOpen) {
    desiredOpen = Math.max(1, Math.min(desiredOpen, lanes.size(
)));
    int count = 0;
    for (Lane l : lanes) {
        l.open = count < desiredOpen;
        if (l.open) count++;
    }
}
public void printLog() {
    sharedLogger.log("\nToll Booth Traffic Log:");
    for (String logEntry : log) sharedLogger.log(logEntry);
}
}
/* =====
* ===== Pricing/Payt. =====
* ===== */
static final class PricingEngine {
    final PricingPolicy policy;
    final BoothLogger log;
    PricingEngine(PricingPolicy p, BoothLogger log) { this.policy =
p; this.log = log; }
    /**
     * Quote a fee per-vehicle:
     *   basePrice * (1 + classMultiplier*(weight-
1)) * surge(traffic) * peak/off-peak factor
     */
    double quote(int minuteOfDay, VehicleClass vClass, int currentT
raffic, int maxTraffic, int openLanes) {
        double fee = policy.basePrice;
        fee *= (1.0 + policy.classMultiplier * (vClass.weight - 1.0
));
        double load = maxTraffic > 0 ? (double) currentTraffic / (d
ouble) maxTraffic : 0.0;
        double surge = 1.0 + policy.surgeK * Math.max(0.0, load - 0
.5); // kick in after 50%
        fee *= surge;

```

```

        if (!policy.isPeak(minuteOfDay)) fee *= (1.0 - policy.offPeakDiscount);
        // Optional: Lane scarcity factor (not too aggressive)
        double scarcity = Math.max(0.9, 1.0 + (Math.max(0, 5 - openLanes) * 0.02));
        fee *= scarcity;
        return Math.max(0.0, fee);
    }
}

/* =====
 * ===== Sensors =====
 * ===== */
interface Sensor<T> { T read(); }
static final class FlowSensor implements Sensor<Integer> {
    private final Random rnd;
    private final int mean;
    private final int spread;
    private final boolean canNeg;
    FlowSensor(long seed, int mean, int spread, boolean canNeg) {
        this.rnd = new Random(seed);
        this.mean = mean; this.spread = Math.max(0, spread); this.canNeg = canNeg;
    }
    @Override public Integer read() {
        int delta = mean + (int)Math.round(rnd.nextGaussian() * spread);
        return canNeg ? delta : Math.abs(delta);
    }
}

static final class SpeedSensor implements Sensor<Double> {
    private final Random rnd;
    private final double mean;
    private final double std;
    SpeedSensor(long seed, double mean, double std) {
        this.rnd = new Random(seed); this.mean = mean; this.std = Math.max(0.0, std);
    }
    @Override public Double read() {
        double v = mean + rnd.nextGaussian() * std;
        return Math.max(0.0, v);
    }
}

static final class QueueSensor implements Sensor<Integer> {
    private final List<Lane> lanes;

```

```

    QueueSensor(List<Lane> lanes) { this.lanes = lanes; }
    @Override public Integer read() {
        int total = 0, open = 0;
        for (Lane l : lanes) if (l.open) { total += Math.max(0, l.queue); open++; }
        return open == 0 ? 0 : total / open;
    }
}
/* =====
 * ===== Controller =====
 * ===== */
static final class TrafficController {
    private final TollBooth booth;
    private final BoothLogger log;
    boolean peakBias = true;
    boolean compressNegSpikes = true;
    TrafficController(TollBooth booth, BoothLogger log) {
        this.booth = booth; this.log = log;
    }
    void setPeakBias(boolean on) { this.peakBias = on; }
    void setCompressNegSpikes(boolean on) { this.compressNegSpikes = on; }
    int preprocessArrive(int req, int minuteOfDay) {
        if (peakBias && isPeak(minuteOfDay) && req > 0) {
            int clipped = (int)Math.round(req * 0.90); // meter arrivals a bit
            if (clipped != req) log.log("[Controller] Peak arrival metering: " + req + " -> " + clipped);
            return clipped;
        }
        if (compressNegSpikes && req < -1000) {
            log.log("[Controller] Compress large negative arrival: " + req + " -> -1000");
            return -1000;
        }
        return req;
    }
    int preprocessExit(int req, int minuteOfDay) {
        if (peakBias && isPeak(minuteOfDay) && req > 0) {
            int boosted = (int)Math.round(req * 1.08); // encourage faster processing
            if (boosted != req) log.log("[Controller] Peak exit boost: " + req + " -> " + boosted);
            return boosted;
        }
    }
}

```

```

    }
    if (compressNegSpikes && req < -1000) {
        log.log("[Controller] Compress large negative exit: " +
req + " -> -1000");
        return -1000;
    }
    return req;
}

private boolean isPeak(int minuteOfDay) {
    int m = minuteOfDay % (24 * 60);
    return (m >= (7 * 60) && m <= (9 * 60 + 30)) || (m >= (16 *
60 + 30) && m <= (19 * 60 + 30));
}
}

/* =====
 * ===== Scenarios =====
 * ===== */
static final class Scenario {
    final String name;
    final List<Runnable> steps = new ArrayList<>();
    Scenario(String name) { this.name = name; }
    Scenario step(Runnable r) { steps.add(r); return this; }
    void run(BoothLogger log) {
        log.log("\n=== Running Scenario: " + name + " (steps=" + st
eps.size() + ") ===");
        for (Runnable r : steps) r.run();
    }
}

static final class ScenarioFactory {
    static Scenario VT(TollBooth t, TrafficController c, BoothLogge
r log) {
        Scenario sc = new Scenario("VT");
        // Your VT arrivals/exits (kept exactly, as provided)
        int[] arr = {
            40,20, /* exit 30 -> we will mix arrivals/exits in orde
r below */
            10, /* exit 50, then the long list you provided... */
            -188,-267,-456,-117,426,-459,107,-50,-317,-
53,70,362,364,-107,-35,-178,409,-166,133,-130,-153,-205,-404,-
325,225,31,317,-141,223,-411,-242,-448,159,-271,-373,482,99,445,-426,-
272,153,-90,-156,15,-42
        };
        int[] ext = {

```

```

        30, 50, -188, -240, 400, -148, -181, -78, -227, -257, -
480, 102, -200, 310, 454, 210, 363, 382, -481, -290, 73, 466, 486, -181, 25, -
314, 292, -307, -191, -18, 381, -77, 145, 377, 39, -382, -409, 440, 448, -104, 8, 261, -
456, -44, -152, -137, -58, 335, -159, -153, -16, -325
    };
    final int[] tmin = { 8*60 }; // 08:00
    // interleave first two special entries/exits to reflect yo
ur header
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(40, tmin[
0]), "TAG-A", VehicleClass.CAR, tmin[0]++));
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(20, tmin[
0]), "TAG-B", VehicleClass.TRUCK, tmin[0]++));
    sc.step(() -> t.vehicleExits(c.preprocessExit(30, tmin[0]),
tmin[0]++));
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(10, tmin[
0]), "TAG-C", VehicleClass.BUS, tmin[0]++));
    sc.step(() -> t.vehicleExits(c.preprocessExit(50, tmin[0]),
tmin[0]++));
    // now the rest of sequence
    for (int i = 4; i < arr.length; i++) {
        int v = arr[i];
        sc.step(() -> t.vehicleArrives(c.preprocessArrive(v, tm
in[0]), null, VehicleClass.CAR, tmin[0]++));
    }
    for (int v : ext) {
        sc.step(() -> t.vehicleExits(c.preprocessExit(v, tmin[0
]), tmin[0]++));
    }
    return sc;
}
static Scenario UVT(TollBooth t, TrafficController c, BoothLogg
er log) {
    // Use exactly your UVT block (which is actually the same a
s Z3 block here)
    Scenario sc = new Scenario("UVT");
    int[] a = {
        40, 20, /* then exit 30 */ 10, /* exit 50 */ -188, -267, -
456, -117, 426, -459, 107, -50, -317, -53, 70, 362, 364, -107, -35, -178, 409, -
166, 133, -130, -153, -205, -404, -325, 225, 31, 317, -141, 223, -411, -242, -
448, 159, -271, -373, 482, 99, 445, -426, -272, 153, -90, -156, 15, -42
    };
    int[] e = {
        30, 50, -188, -240, 400, -148, -181, -78, -227, -257, -480, 102, -
200, 310, 454, 210, 363, 382, -481, -290, 73, 466, 486, -181, 25, -314, 292, -307, -

```

```

191,-18,381,-77,145,377,39,-382,-409,440,448,-104,8,261,-456,-44,-152,-
137,-58,335,-159,-153,-16,-325
    };
    final int[] tmin = { 17*60 }; // 17:00
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(40, tmin[
0]), "TAG-U1", VehicleClass.CAR, tmin[0]++));
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(20, tmin[
0]), "TAG-U2", VehicleClass.CAR, tmin[0]++));
    sc.step(() -> t.vehicleExits(c.preprocessExit(30, tmin[0]),
tmin[0]++));
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(10, tmin[
0]), "TAG-U3", VehicleClass.MOTORCYCLE, tmin[0]++));
    sc.step(() -> t.vehicleExits(c.preprocessExit(50, tmin[0]),
tmin[0]++));
    for (int i = 4; i < a.length; i++) {
        int v = a[i];
        sc.step(() -> t.vehicleArrives(c.preprocessArrive(v, tm
in[0]), null, VehicleClass.CAR, tmin[0] += 2));
    }
    for (int v : e) {
        sc.step(() -> t.vehicleExits(c.preprocessExit(v, tmin[0
]), tmin[0] += 2));
    }
    return sc;
}
static Scenario Z3(TollBooth t, TrafficController c, BoothLogge
r log) {
    // Same sequence as your Z3 comment block
    Scenario sc = new Scenario("Z3");
    int[] a = {
        40,20, /* exit 30 */ 10, /* exit 50 */ -188,-267,-456,-
117,426,-459,107,-50,-317,-53,70,362,364,-107,-35,-178,409,-166,133,-
130,-153,-205,-404,-325,225,31,317,-141,223,-411,-242,-448,159,-271,-
373,482,99,445,-426,-272,153,-90,-156,15,-42
    };
    int[] e = {
        30,50,-188,-240,400,-148,-181,-78,-227,-257,-480,102,-
200,310,454,210,363,382,-481,-290,73,466,486,-181,25,-314,292,-307,-
191,-18,381,-77,145,377,39,-382,-409,440,448,-104,8,261,-456,-44,-152,-
137,-58,335,-159,-153,-16,-325
    };
    final int[] tmin = { 6*60 + 30 }; // 06:30
    sc.step(() -> t.vehicleArrives(c.preprocessArrive(40, tmin[
0]), "TAG-Z1", VehicleClass.CAR, tmin[0]++));

```

```

        sc.step(() -> t.vehicleArrives(c.preprocessArrive(20, tmin[0]), "TAG-Z2", VehicleClass.TRUCK, tmin[0]++));
        sc.step(() -> t.vehicleExits(c.preprocessExit(30, tmin[0]), tmin[0]++));
        sc.step(() -> t.vehicleArrives(c.preprocessArrive(10, tmin[0]), "TAG-Z3", VehicleClass.BUS, tmin[0]++));
        sc.step(() -> t.vehicleExits(c.preprocessExit(50, tmin[0]), tmin[0]++));
        for (int i = 4; i < a.length; i++) {
            int v = a[i];
            sc.step(() -> t.vehicleArrives(c.preprocessArrive(v, tmin[0]), null, VehicleClass.CAR, tmin[0]++));
        }
        for (int v : e) {
            sc.step(() -> t.vehicleExits(c.preprocessExit(v, tmin[0]), tmin[0]++));
        }
        return sc;
    }

    static Scenario FT(TollBooth t, TrafficController c, BoothLogger log, long seed) {
        Scenario sc = new Scenario("FT");
        Random r = new Random(seed);
        final int[] m = { 12 * 60 }; // noon
        // random arrivals/exits with possible large magnitudes (stress)
        for (int i = 0; i < 120; i++) {
            int arr = r.nextBoolean() ? r.nextInt(1500) : -r.nextInt(1500);
            sc.step(() -> t.vehicleArrives(c.preprocessArrive(arr, m[0]), null, VehicleClass.CAR, m[0]++));
            int ext = r.nextInt(1200) - r.nextInt(1200);
            sc.step(() -> t.vehicleExits(c.preprocessExit(ext, m[0]), m[0]++));
        }
        return sc;
    }
}

/* =====
 * ===== Runner/CLI =====
 * ===== */
enum Mode { VT, FT, Z3, UVT }
static final class Runner {
    private final TollBooth booth;

```

```

        private final BoothLogger log;
        Runner(TollBooth booth, BoothLogger log) {
            this.booth = booth; this.log = log;
        }
        void run(Scenario sc) {
            log.log("Open lanes (start): " + booth.getOpenLanes() + ", Traffic=" + booth.getCurrentTraffic());
            sc.run(log);
            log.log("Open lanes (end): " + booth.getOpenLanes() + ", Traffic=" + booth.getCurrentTraffic());
        }
        void printFinal() {
            booth.stats().print(log);
            log.printSummary();
            // booth.printLog(); // optionally dump raw lines
        }
    }
    /* =====
    * ===== Health Checks =====
    * ===== */
    static final class Health {
        static void checkPolicy(LanePolicy p, BoothLogger log) {
            if (p.highRatio <= 0 || p.highRatio > 1.0) log.alert(AlertType.INFO, "High ratio out of (0,1]: " + p.highRatio);
            if (p.safetyMin <= 0) log.alert(AlertType.INFO, "Safety min should be > 0");
        }
        static void checkPricing(PricingPolicy p, BoothLogger log) {
            if (p.basePrice < 0) log.alert(AlertType.INFO, "Base price < 0");
            if (p.offPeakDiscount < 0 || p.offPeakDiscount > 0.9) log.alert(AlertType.INFO, "Off-peak discount suspicious: " + p.offPeakDiscount);
        }
    }
    /* =====
    * ===== Main =====
    * ===== */
    public static void main(String[] args) {
        // Defaults
        Mode mode = Mode.UVT;
        long seed = 20251104L; // deterministic seed (Tokyo date)
        boolean consoleLog = true; // LOG=on/off
    }

```

```

        boolean pricingOn = true;        // PRICING=on/off
        // Parse CLI
        for (String a : args) {
            String s = a.trim();
            if (s.startsWith("MODE=")) {
                String m = s.substring("MODE=".length()).toUpperCase(Locale.ROOT);

                try { mode = Mode.valueOf(m); } catch (Exception ignore) {}

            } else if (s.startsWith("SEED=")) {
                try { seed = Long.parseLong(s.substring("SEED=".length())); } catch (Exception ignore) {}
            } else if (s.startsWith("LOG=")) {
                String v = s.substring("LOG=".length()).toLowerCase(Locale.ROOT);

                consoleLog = v.equals("on") || v.equals("true") || v.equals("1");
            } else if (s.startsWith("PRICING=")) {
                String v = s.substring("PRICING=".length()).toLowerCase(Locale.ROOT);

                pricingOn = v.equals("on") || v.equals("true") || v.equals("1");
            }
        }

        BoothLogger logger = new BoothLogger();
        logger.setConsole(consoleLog);
        // Lane policy mirrors your thresholds: >80% -> 8 Lanes, <minTraffic -> 3 Lanes, otherwise 5; safety min=2; anti-thrash=5min
        LanePolicy lanePolicy = new LanePolicy(
            0.80, // high ratio
            8,    // high open
            5,    // normal open
            3,    // low open
            2,    // safety min Lanes
            5     // anti-thrash window (minutes)
        );

        // Pricing policy (optional)
        PricingEngine pricing = null;
        if (pricingOn) {
            PricingPolicy pp = new PricingPolicy(
                2.50, // base price
                0.40, // class multiplier
                0.85, // surge coefficient
                0.25, // off-peak discount
            );
        }
    }
}

```

```

        7*60, 9*60+30,    // morning peak 07:00-09:30
        17*60, 19*60+30 // evening peak 17:00-19:30
    );
    Health.checkPricing(pp, logger);
    pricing = new PricingEngine(pp, logger);
}
// Build booth (your baseline: max=100, min=30, initial=50)
TollBooth booth = new TollBooth(100, 30, 50, lanePolicy, pricing, logger);
// Open some demo accounts for EZPass Lanes
booth.openAccount("TAG-A", 50.00);
booth.openAccount("TAG-B", 20.00);
booth.openAccount("TAG-C", 15.00);
booth.openAccount("TAG-U1", 12.50);
booth.openAccount("TAG-U2", 16.00);
booth.openAccount("TAG-U3", 7.25);
booth.openAccount("TAG-Z1", 25.00);
booth.openAccount("TAG-Z2", 30.00);
booth.openAccount("TAG-Z3", 4.00);
Health.checkPolicy(lanePolicy, logger);
TrafficController controller = new TrafficController(booth, logger);
controller.setPeakBias(true);
controller.setCompressNegSpikes(true);
Runner runner = new Runner(booth, logger);
// Choose scenario based on your provided sequences
Scenario sc;
switch (mode) {
    case VT:
        sc = ScenarioFactory.VT(booth, controller, logger);
        break;
    case Z3:
        sc = ScenarioFactory.Z3(booth, controller, logger);
        break;
    case FT:
        sc = ScenarioFactory.FT(booth, controller, logger, seed);
        break;
    default:
        // sc = ScenarioFactory.UVT(booth, controller, logger);
        sc = ScenarioFactory.FT(booth, controller, logger, seed);
}
logger.log("=== Smart Toll Booth System (Mode=" + mode + ", Seed=" + seed + ", Pricing=" + (pricingOn?"on":"off") + ") ===");

```

```

        runner.run(sc);
        runner.printFinal();
        // Final tiny sensor sanity check (informational)
        SpeedSensor sp = new SpeedSensor(seed ^ 0xABCDEF, 68.0, 9.5);
        double v = sp.read();
        if (Double.isNaN(v) || v < 0) logger.alert(AlertType.SENSOR_FAULT, "SpeedSensor abnormal: " + v);
        else logger.log(String.format(Locale.ROOT, "SpeedSensor sample: %.1f km/h", v));
    }
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c124_SmartTollBoothSystemEx.java	46.6 %	81	93	174
▼ c124_SmartTollBoothSystemEx	17.6 %	6	28	34
main(String[])	17.6 %	6	28	34
> ScenarioFactory	25.0 %	4	12	16
> Health	50.0 %	6	6	12
> PricingEngine	50.0 %	2	2	4
> PricingPolicy	50.0 %	4	4	8
> TagAccount	50.0 %	1	1	2
> TollBooth	59.5 %	25	17	42
> TrafficController	62.5 %	20	12	32
> BoothLogger	75.0 %	6	2	8
> LanePolicy	83.3 %	5	1	6
> Scenario	100.0 %	2	0	2
> Alert	100.0 %	0	0	0
> AlertType	100.0 %	0	0	0

3. Vibration testing instruction coverage: 89%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c124_SmartTollBoothSystemEx.java	55.2 %	2,142	1,741	3,883
▼ c124_SmartTollBoothSystemEx	61.5 %	195	122	317
> ScenarioFactory	31.6 %	626	1,353	1,979
> Health	56.6 %	30	23	53
main(String[])	62.1 %	195	119	314
> TollBooth	78.4 %	429	118	547
> TrafficController	89.0 %	113	14	127
> PricingEngine	89.5 %	77	9	86
> LanePolicy	90.2 %	55	6	61
> TagAccount	91.7 %	22	2	24
> BoothLogger	94.0 %	125	8	133
> PricingPolicy	96.1 %	49	2	51
> Alert	100.0 %	21	0	21
> AlertType	100.0 %	84	0	84
> BoothStats	100.0 %	84	0	84

4. Fuzz testing branch coverage: 37.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c124_SmartTollBoothSystemEx	17.6 %	6	28	34
> TagAccount	0.0 %	0	2	2
main(String[])	17.6 %	6	28	34
> ScenarioFactory	25.0 %	4	12	16
> PricingPolicy	37.5 %	3	5	8
> TrafficController	37.5 %	12	20	32
> Health	50.0 %	6	6	12
> PricingEngine	50.0 %	2	2	4
> TollBooth	52.4 %	22	20	42
> LanePolicy	66.7 %	4	2	6
> BoothLogger	75.0 %	6	2	8
> Scenario	100.0 %	2	0	2
> Alert		0	0	0
> AlertType		0	0	0
> BoothStats		0	0	0

5. Fuzz testing instruction coverage: 59.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c124_SmartTollBoothSystemEx	61.6 %	196	122	318
> ScenarioFactory	5.0 %	98	1,881	1,979
> TagAccount	37.5 %	9	15	24
> Health	56.6 %	30	23	53
> TrafficController	59.1 %	75	52	127
main(String[])	62.2 %	196	119	315
> TollBooth	70.2 %	384	163	547
> LanePolicy	80.3 %	49	12	61
> PricingPolicy	88.2 %	45	6	51
> BoothLogger	94.0 %	125	8	133
> PricingEngine	98.8 %	85	1	86
> Alert	100.0 %	21	0	21
> AlertType	100.0 %	84	0	84
> BoothStats	100.0 %	84	0	84
> Lane	100.0 %	15	0	15

6. Z3 branch coverage: 37.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c124_SmartTollBoothSystemEx.java	38.5 %	67	107	174
▼ c124_SmartTollBoothSystemEx	17.6 %	6	28	34
> TagAccount	0.0 %	0	2	2
main(String[])	17.6 %	6	28	34
> ScenarioFactory	25.0 %	4	12	16
> PricingPolicy	37.5 %	3	5	8
> TrafficController	37.5 %	12	20	32
> Health	50.0 %	6	6	12
> PricingEngine	50.0 %	2	2	4
> TollBooth	52.4 %	22	20	42
> LanePolicy	66.7 %	4	2	6
> BoothLogger	75.0 %	6	2	8
> Scenario	100.0 %	2	0	2
> Alert		0	0	0
> AlertType		0	0	0

7. Z3 instruction coverage: 59.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c124_SmartTollBoothSystemEx.java	39.1 %	1,517	2,367	3,884
▼ c124_SmartTollBoothSystemEx	61.6 %	196	122	318
> ScenarioFactory	5.0 %	98	1,881	1,979
> TagAccount	37.5 %	9	15	24
> Health	56.6 %	30	23	53
> TrafficController	59.1 %	75	52	127
main(String[])	62.2 %	196	119	315
> TollBooth	70.2 %	384	163	547
> LanePolicy	80.3 %	49	12	61
> PricingPolicy	88.2 %	45	6	51
> BoothLogger	94.0 %	125	8	133
> PricingEngine	98.8 %	85	1	86
> Alert	100.0 %	21	0	21
> AlertType	100.0 %	84	0	84
> BoothStats	100.0 %	84	0	84

8. Unimproved V-method branch coverage: 62.5%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c124_SmartTollBoothSystemEx.java		46.6 %	81	93	174
▼ c124_SmartTollBoothSystemEx		17.6 %	6	28	34
main(String[])		17.6 %	6	28	34
> ScenarioFactory		25.0 %	4	12	16
> Health		50.0 %	6	6	12
> PricingEngine		50.0 %	2	2	4
> PricingPolicy		50.0 %	4	4	8
> TagAccount		50.0 %	1	1	2
> TollBooth		59.5 %	25	17	42
▼ TrafficController		62.5 %	20	12	32
▲ preprocessArrive(int, int)		58.3 %	7	5	12
■ isPeak(int)		62.5 %	5	3	8
▲ preprocessExit(int, int)		66.7 %	8	4	12
▲ TrafficController(TollBooth, Booth			0	0	0
▲ setCompressNegSpikes(boolean)			0	0	0

9. Unimproved V-method instruction coverage: 89%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c124_SmartTollBoothSystemEx.java		55.2 %	2,142	1,741	3,883
▼ c124_SmartTollBoothSystemEx		61.5 %	195	122	317
> ScenarioFactory		31.6 %	626	1,353	1,979
> Health		56.6 %	30	23	53
main(String[])		62.1 %	195	119	314
> TollBooth		78.4 %	429	118	547
▼ TrafficController		89.0 %	113	14	127
▲ preprocessArrive(int, int)		83.3 %	35	7	42
▲ preprocessExit(int, int)		83.3 %	35	7	42
▲ TrafficController(TollBooth, Booth		100.0 %	15	0	15
■ isPeak(int)		100.0 %	20	0	20
▲ setCompressNegSpikes(boolean)		100.0 %	4	0	4
▲ setPeakBias(boolean)		100.0 %	4	0	4
> PricingEngine		89.5 %	77	9	86
> LanePolicy		90.2 %	55	6	61